



CT126-3-2-CTF-PRACTICAL CTF STRATEGIES

GROUP ASSIGNMENT (40%)



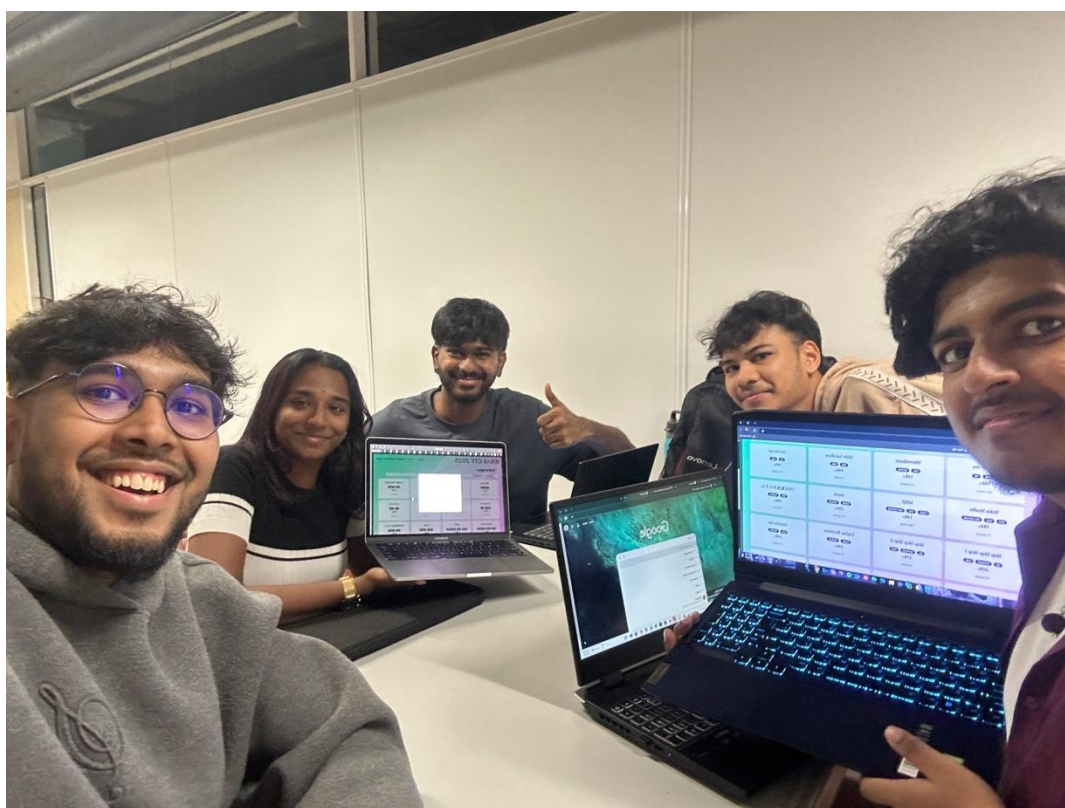
IERAE CTF 2025

GROUP NAME:	DUTCH_LADY	
TP NUMBER	NAME	WORKLOAD
TP070844	HEMANTHRAJAH A/L N KRISHNARAJAH	20%
TP071154	ANUSHA DEVY A/P BRAHMATARAN	20%
TP070071	DARMIK ISHWAR A/L RAVI	20%
TP071194	ADAM JEROME JOSEPH	20%
TP070996	SAILESS RAJ A/L RAJENDRAN	20%

1.0 Workload Matrix

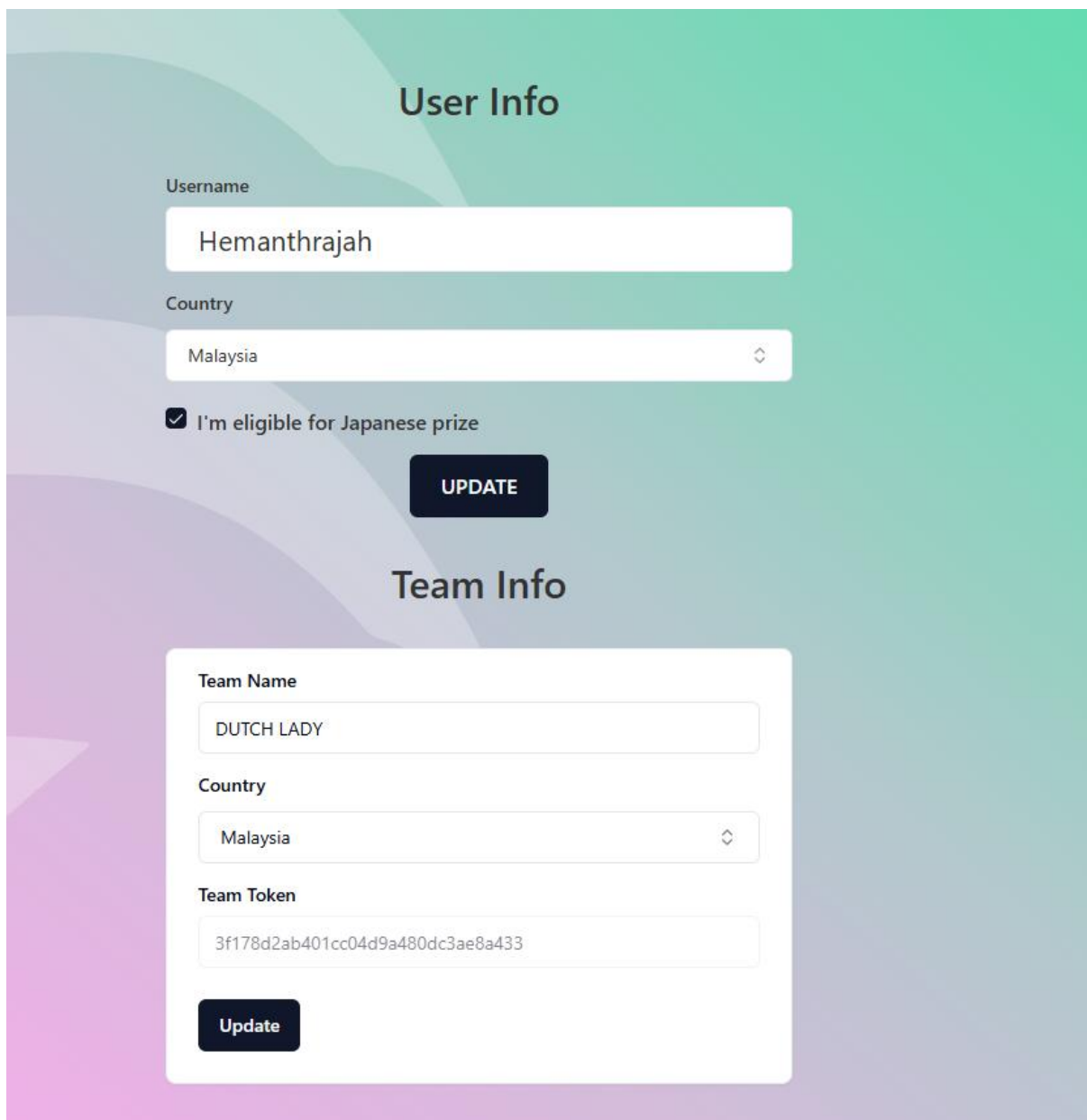
NAME	TP NUMBER	WORKLOAD
HEMANTHRAJAH A/L N KRISHNARAJAH	TP070844	Tools, Commands, Scripts
ANUSHA DEVY A/P BRAHMATARAN	TP071154	Write-up Content
DARMIK ISHWAR A/L RAVI	TP070071	Conclusion
ADAM JEROME JOSEPH	TP071194	Vulnerabilities and Skills Gained
SAILESS RAJ A/L RAJENDRAN	TP070996	Challenges and Lesson Learnt

2.0 Group Selfies





3.0 Write-up Content



The image shows a web interface for updating user and team information. The background is a gradient of teal, purple, and pink. The 'User Info' section is at the top, followed by the 'Team Info' section.

User Info

Username
Hemanthrajah

Country
Malaysia

☒ I'm eligible for Japanese prize

UPDATE

Team Info

Team Name
DUTCH LADY

Country
Malaysia

Team Token
3f178d2ab401cc04d9a480dc3ae8a433

Update

Figure 1: IERAE CTF 2025 Team info

253	the_normal	398	🚩	🚩		🚩	🚩												
254	nop_team	398	🚩	🚩		🚩	🚩												
255	shikasan	398	🚩	🚩		🚩	🚩												
256	Cyb3r Mischief	398	🚩	🚩		🚩	🚩												
257	DOUBLE STANDARD	398	🚩	🚩		🚩	🚩												
258	HackSick	398	🚩	🚩		🚩	🚩												
259	FlyingMelonPan	398	🚩	🚩		🚩	🚩												
260	y0d3n ♡	398	🚩	🚩		🚩	🚩												
261	Ex0rcists	398	🚩	🚩		🚩	🚩												
262	DUTCH LADY	398	🚩	🚩		🚩	🚩												
263	FPTU Ethical Hackers Club	398	🚩	🚩		🚩	🚩												
264	SAKURAHack	398	🚩	🚩		🚩	🚩												
265	MySelf	398	🚩	🚩		🚩	🚩												

Figure 2: DUTCH LADY scoreboard (top 35%)

This document presents the solution write-up for the CTF challenge titled **Rev Rev Rev**, that was solved during the **I ERAE CTF 2025** online competition. This was a live Capture The Flag (CTF) event that tested various cybersecurity and reverse engineering skills. Our team selected this challenge as part of the Reverse Engineering category. It was worth 102 points and was marked as medium difficulty. The challenge involved understanding a Python script used to transform a flag and required reversing the transformation to retrieve the original flag. The steps involved multiple layers of logical reasoning, binary operations, and code analysis.

Upon downloading the challenge, we received two files that are `chal.py` and `output.txt`.

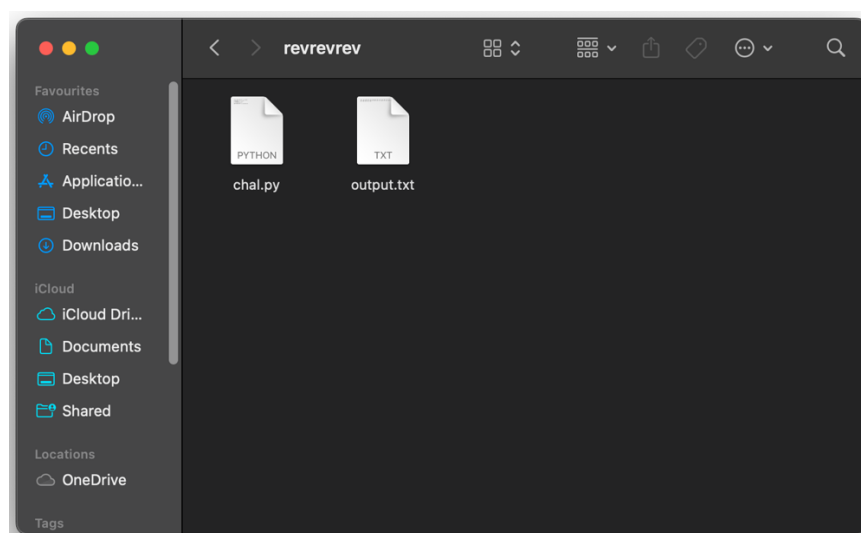


Figure 3: Shows Content of File

The chal.py script was responsible for disguising the original flag. The purpose was to analyse and reverse the transformation process using only the data from output.txt, as the original flag file “flag.txt” was not provided.

The contents of chal.py were as follows:

```
1  flag = open('flag.txt', 'r').read()
2  x = [ord(c) for c in flag]
3  x.reverse()
4  y = [i^0xff for i in x]
5  z = [~i for i in y]
6  open('output.txt', 'w').write(str(z))
```

Figure 4: Shows Content of “chal.py”

The script performs a multi-step transformation process to obscure the original flag. First, it reads the flag from a text file called flag.txt and stores it as a regular string. This string is then converted into a list of ASCII values using the ord() function, which transforms each character into its corresponding numerical code for example, 'A' becomes 65. Once the list of ASCII codes is created, it is reversed using x.reverse(), changing the order of the characters so that the first becomes last and vice versa. After the reversal, each number in the list is subjected to a bitwise XOR operation with 0xff which is 255 in decimal. XOR with 255 effectively inverts all bits in an 8-bit binary representation, so a value like 65 in binary 01000001 becomes 190 in binary 10111110. This adds a layer of obfuscation by flipping every bit in the number. Finally, the script applies a bitwise NOT operation (~i) to each of these XORed values. In Python, the bitwise NOT operator returns the complement of the number (i.e., ~x = -(x+1)), which typically results in negative integers when applied to positive values. These negative numbers are then stored in a new list and written to a file named “output.txt”. This entire process makes the original flag unrecognizable, ensuring that only someone who understands the transformation logic can reverse it and recover the original string.

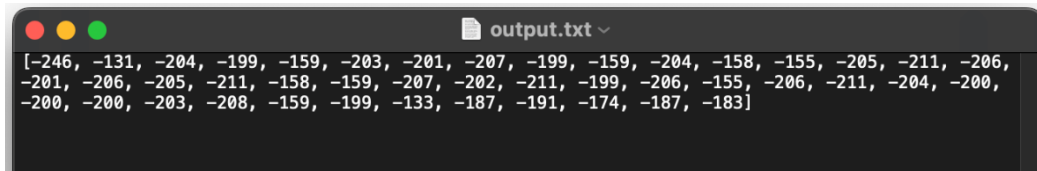


Figure 5: Shows Content of “output.txt”

To recover the original flag, we must undo each transformation step in reverse order. Firstly, we have to read the list of integers from output.txt. Then, we apply the bitwise NOT operation again to reverse the last transformation. XOR each resulting integer with 0xff to revert the previous operation and reverse the list. Convert the ASCII values back to characters using chr().

We created a new Python file solve.py and used the following script to reverse the process:

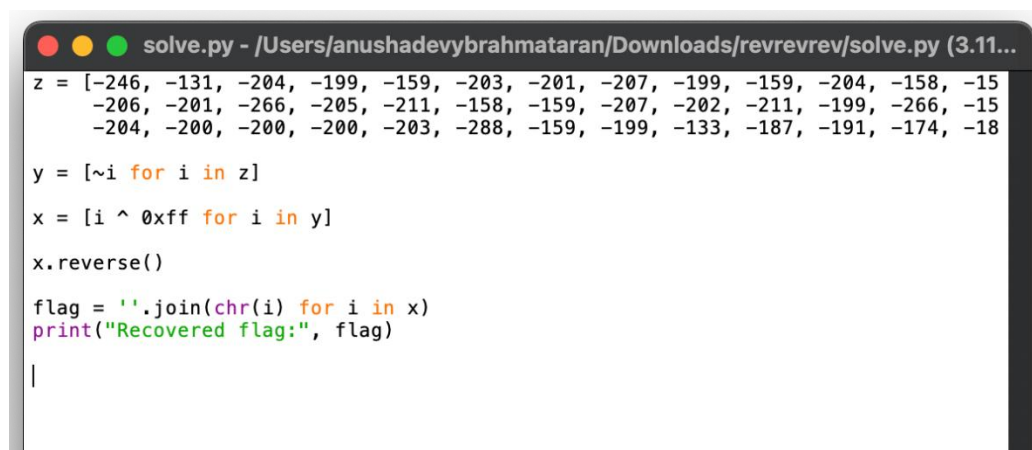
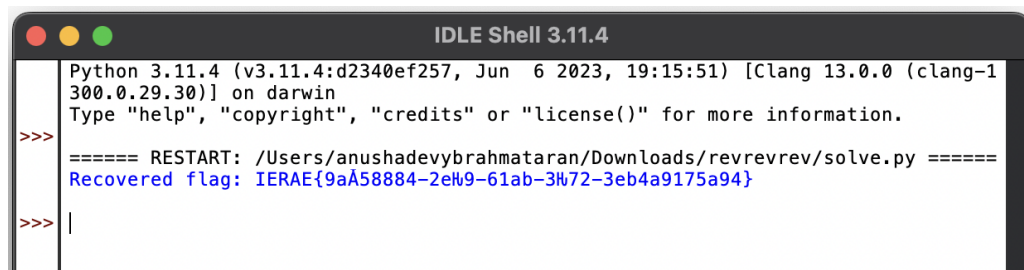


Figure 6: Shows Implementation of Codes in “solve.py”

The given Python script is designed to reverse a transformation applied to a flag during a CTF reverse engineering challenge. The variable z holds a list of negative integers that represent the disguised ASCII values of the flag's characters. The first step, `y = [~i for i in z]`, applies a bitwise NOT operation to each integer in z, effectively undoing the last transformation that was originally applied in the encryption script. Next, the line `x = [i ^ 0xff for i in y]` performs a bitwise XOR with 0xff 255 in decimal on each element in y, which reverses the previous XOR operation used during encryption. After that, `x.reverse()` restores the original order of the list since the initial obfuscation involved reversing the character order. Finally, `flag = ".join(chr(i) for i in x)` converts each integer back into its corresponding ASCII character,

and `print("Recovered flag:", flag)` outputs the reconstructed flag string. This script effectively decodes the obfuscated data and recovers the original CTF flag.

Upon running the script, the following output was obtained:



```
Python 3.11.4 (v3.11.4:d2340ef257, Jun 6 2023, 19:15:51) [Clang 13.0.0 (clang-1300.0.29.30)] on darwin
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: /Users/anushadevybrahmataran/Downloads/revrevrev/solve.py =====
Recovered flag: IERAE{9aA58884-2eW9-61ab-3W72-3eb4a9175a94}
>>> |
```

Figure 7: Shows the Output of the Code that is the Flag

Recovered flag: **IERAE{9a058884-2e29-61ab-3272-3eb4a9175a94}**

This confirmed the correct reversal of the transformations and successful flag retrieval.

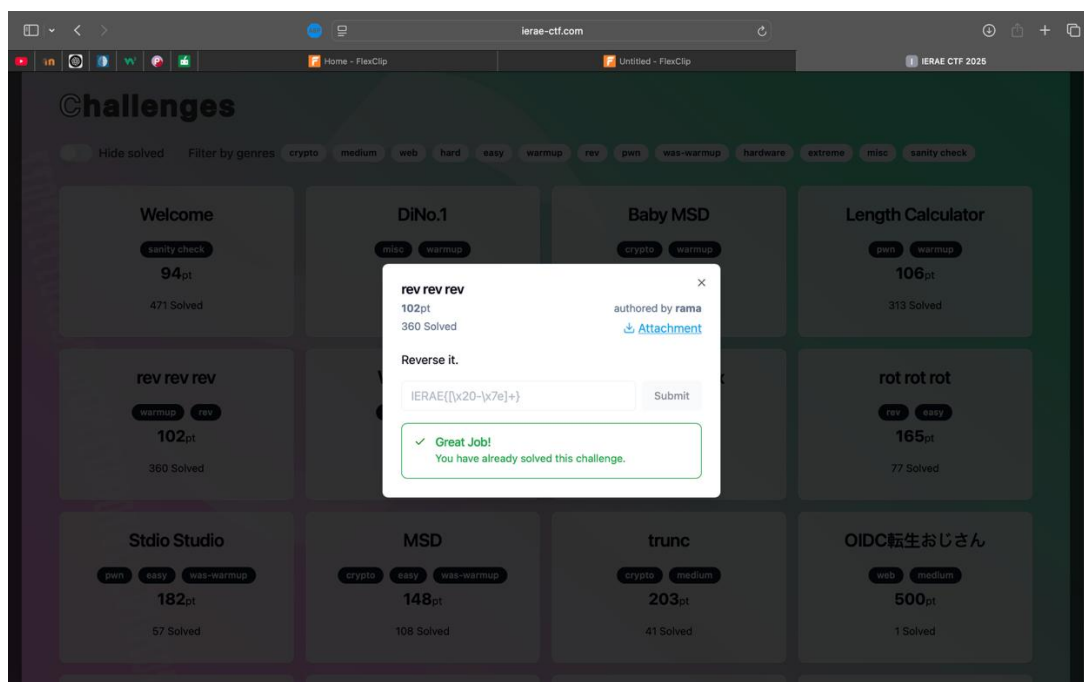


Figure 8: Shows Successful Completion of the Challenge

3.1 Tools Commands, and Scripts Used



Figure 9: Shows Python3 tool that was Used in CTF challenge

The primary tool used for solving this challenge was Python 3, chosen because the original transformation script (`chal.py`) was written in Python. This made it logical and efficient to implement the reverse logic in the same language. Python provides built-in support for bitwise operations (`^`, `~`), list manipulation (`reverse()`), and character encoding (`ord()` and `chr()`), all of which were essential for both understanding and reversing the transformations (Python Software Foundation, 2024).

To test and run the script, we used the Python interactive shell (REPL) and IDLE, Python's built-in development environment. These tools allowed us to write, test, and debug the decryption logic efficiently. The REPL was particularly helpful for experimenting with how Python handles operations like `~i` (bitwise NOT) and `i ^ 0xff` (bitwise XOR) on individual values before scaling the logic to the full list. Each tool contributed to a lightweight and focused workflow tailored for scripting and testing small-scale code quickly.

3.2 Vulnerabilities and Skills Gained

This challenge did not involve traditional vulnerabilities like buffer overflows, injections, or memory corruption. Instead, it focused on logic-based obfuscation through reversible transformations. The challenge simulated a scenario where sensitive data in this case, a flag was deliberately obscured using basic cryptographic operations (Real Python, 2023). Understanding the use of bitwise XOR with 0xff to invert bits and bitwise NOT to flip bit values and invert signs was crucial. These are common in both encryption routines and anti-debugging logic, making them highly relevant in reverse engineering and binary analysis contexts (Zeng & Kumar, 2021).

By completing this challenge, our team enhanced key reverse engineering skills, including reading and analyzing code, identifying the purpose of obfuscation layers, and manually reconstructing the original logic. We also reinforced our understanding of ASCII encoding, list handling, and how bitwise operations work at the binary level (Li & Rao, 2023). These skills are foundational in fields such as malware analysis, exploit development, and debugging compiled binaries.

3.3 Challenges, Experiences, and Lessons Learned as Beginners

As beginners in a live CTF environment, one of our biggest initial challenges was interpreting the purpose of each line in the `chal.py` script. While the syntax was simple, understanding the effect of bitwise operations on ASCII characters took some effort. We encountered confusion around how the bitwise NOT (`~`) operation produced negative numbers and how Python handled bit representation internally. To overcome this, we conducted small-scale tests in the Python shell, working with familiar characters like 'A' or 'Z' to observe how the transformations behaved.

Team collaboration was key to solving the challenge. One member focused on testing and decoding logic step-by-step, while another member documented findings and built the final decryption script. Regular check-ins helped us share observations and refine our approach together. Despite facing time pressure and occasional logic errors, we maintained clear communication and divided tasks based on each member's strengths.

Participating in this CTF helped us learn the importance of reversing logic in the correct order, attention to detail, and testing each step before applying it at scale. It also emphasized that not all CTF challenges are about exploitation, some are about thinking logically, reading code carefully, and trusting your debugging process. We finished this challenge not only with a solved flag but also with a better grasp of Python, a deeper understanding of bitwise logic, and the confidence to take on harder reverse engineering problems in the future.

4.0 Conclusion

The **Rev Rev Rev** challenge from the **IERAE CTF 2025** provided a valuable opportunity to apply reverse engineering concepts in a practical, hands-on scenario. By carefully analyzing the logic behind the original Python script and systematically reversing each transformation, we were able to successfully recover the hidden flag. Although the challenge did not involve traditional vulnerabilities, it tested our understanding of bitwise operations, ASCII encoding, and logical reversal skills that are essential in many real-world reverse engineering and forensic tasks. Completing this challenge strengthened our ability to break down obfuscated code, reason through multi-step transformations, and implement custom scripts to reverse engineer data. As beginners in a live CTF environment, this experience significantly boosted our confidence and problem-solving mindset for future competitions.

5.0 References

1. What's New in Python. (2025). Python Documentation.
<https://docs.python.org/3.12/whatsnew/index.html>
2. Bartosz Zaczynski. (2020, December 9). Bitwise Operators in Python.
Realpython.com; Real Python. <https://realpython.com/python-bitwise-operators/>
3. IERAE CTF 2025 Writeup. (2025, June 22). Zenn.
<https://zenn.dev/claustra01/articles/b157d0cb6b3eea>
4. Ierae CTF 2025 Writeup. (n.d.). Retrieved from
<https://qiita.com/shamoji8/items/71c649a3e650d7be1c78>
5. Engineer飯沼翼Engineer衣笠公陽, E. (n.d.). IERAE CTF 2025 Writeup - HACK
The Nikkei. Retrieved from https://hack.nikkei.com/blog/ctf_ierae_2025/
6. Zaczynski, B. (2023). Bitwise Operators in Python. Retrieved from
<https://realpython.com/python-bitwise-operators/>
7. W3Schools.com. (n.d.). Retrieved from
https://www.w3schools.com/python/gloss_python_bitwise_operators.asp

8. Coding, P. in P. /, About Author Gabriel RamugliaGabriel is the owner and founder of IOFLOOD.com, & Posts, R. (2024). XOR in Python: Usage Guide to Bitwise XOR. Retrieved from <https://ioflood.com/blog/xor-in-python-usage-guide-to-bitwise-xor/>
9. kumar, A. (2022). What is XOR in Python? Retrieved from <https://www.scaler.com/topics/xor-in-python/>
10. Nowhere to Hide: Three methods of XOR obfuscation. (2024). Retrieved from <https://www.threatdown.com/blog/nowhere-to-hide-three-methods-of-xor-obfuscation/>